

AgroVision

Identificacao de Doencas Foliares do Milho com IA Auditavel

Documentacao tecnica e cientifica do projeto

Trabalho de Conclusao de Curso
Engenharia de Software / Sistemas de Informacao

Luiz Carlos Junior & Equipe AgroVision
Maio de 2026

1. Sumario Executivo

O **AgroVision** é um aplicativo móvel desenvolvido em React Native que utiliza Inteligência Artificial para identificar doenças em folhas de milho (*Zea mays* L.) a partir de uma fotografia. O diferencial do projeto é a inclusão de uma **camada de auditoria visual** baseada em Grad-CAM e duas métricas quantitativas inspiradas em literatura de Explainable AI (XAI): **Attention Leakage (AL)** e **Attention Focus Score (AFS)**. Essas métricas permitem ao usuário não apenas receber uma classificação, mas também entender em quais regiões da folha o modelo se baseou para decidir e quanto dessa atenção está efetivamente sobre a região de interesse.

O projeto cobre quatro classes-alvo: **Helmintosporiose** (*Exserohilum turcicum*), **Ferrugem-Comum** (*Puccinia sorghi*), **Mancha-Cinzenta** (*Cercospora zeae-maydis*) e **Milho Saudável**. O backend de inferência está hospedado no Railway (FastAPI + WebSocket), com o modelo de identificação servido pelo Roboflow Serverless API e o modelo de auditoria (Grad-CAM) executado localmente em TensorFlow/Keras 3.

2. Motivacao e Contexto

2.1. O problema no campo

Doenças foliares causam perdas significativas na produção mundial de milho. Estudos da Embrapa indicam que doenças como ferrugem-comum, helmintosporiose e mancha-cinzenta podem reduzir a produtividade em até 60% quando não detectadas e tratadas no estágio inicial. O diagnóstico tradicional depende de visita técnica e inspeção visual presencial, o que é custoso e demorado para o pequeno e médio produtor.

Aplicativos baseados em redes neurais convolucionais (CNNs) já foram propostos como solução para esse problema. No entanto, a maioria oferece apenas o resultado da classificação, sem permitir que o agricultor entenda *por que* o sistema chegou a aquele diagnóstico. Esse comportamento de "caixa-preta" gera desconfiança legítima por parte do usuário final, que muitas vezes precisa tomar decisões financeiras (aplicar fungicida, replantar) com base naquele resultado.

2.2. A dor que o AgroVision resolve

O AgroVision endereça três dores específicas:

- **Falta de transparencia** em modelos de IA agrícola: o usuário não consegue inspecionar a decisão do modelo.
- **Dificuldade de validacao cientifica**: pesquisadores e técnicos agrônomos precisam de métricas para avaliar se o modelo realmente "olha o lugar certo".
- **Limitacao do diagnostico remoto**: regiões rurais com pouca cobertura técnica precisam de uma ferramenta de triagem confiável antes de mobilizar um agrônomo.

2.3. Idealizacao

A proposta nasceu da combinacao de dois conhecimentos: visao computacional aplicada e a necessidade pratica do agronegocio brasileiro de ferramentas de diagnostico foliar precisas e **explicaveis**. O recorte para milho foi escolhido por tres motivos: (i) e a segunda cultura mais produzida no Brasil; (ii) ha disponibilidade publica de datasets anotados de qualidade (PlantVillage e PRMSU); (iii) o numero reduzido de classes (4) permite que o estudo seja executavel no escopo de um TCC sem perder rigor cientifico.

3. Objetivos

3.1. Objetivo geral

Desenvolver um sistema movel de identificacao de doencas foliares no milho que integre, alem da classificacao, uma camada de auditoria visual e quantitativa do modelo de IA, permitindo ao usuario validar a confiabilidade da resposta antes de agir.

3.2. Objetivos especificos

- Construir um pipeline movel completo (captura -> classificacao -> diagnostico) usando arquitetura mobile-first.
- Treinar e servir um modelo de classificacao de quatro classes (Blight, Common_Rust, Gray_Leaf_Spot, Healthy).
- Implementar Grad-CAM (Selvaraju et al., 2017) para geracao de mapas de saliencia visuais.
- Definir e implementar as metricas Attention Leakage (AL) e Attention Focus Score (AFS) como instrumento de auditoria do modelo.
- Garantir reproduzibilidade cientifica: dataset publico, codigo versionado, dependencias fixadas, modelo exportavel.
- Oferecer uma experiencia de usuario simples o suficiente para ser usada em campo, sem treinamento previo.

4. Stack Tecnologica

4.1. Frontend (app movel)

Camada	Tecnologia	Funcao
UI/Runtime	React Native (Expo SDK 52)	Aplicativo iOS/Android single-codebase
Linguagem	TypeScript	Tipagem estatica e contratos claros entre camadas
Navegacao	React Navigation (native-stack)	Stack de telas: Home, Camera, Result, AuditarModelo
Estado	Context API (AnalysisContext)	Gerenciamento de estado de analise em andamento
Camera	expo-camera	Captura de foto em alta resolucao
FS	expo-file-system	Leitura/escrita base64 e cache temporario
Share	expo-sharing	Exportacao do heatmap pos-auditoria
Storage local	SQLite (expo-sqlite)	Historico de analises offline

4.2. Backend (servico de IA)

Camada	Tecnologia	Funcao
API	FastAPI 0.110 + Uvicorn	REST + WebSocket para progresso em tempo real
ML runtime	TensorFlow 2.20 (CPU) + Keras 3.1.2	Inferencia do modelo de auditoria
Visao	OpenCV (cv2)	Pre-processamento, mascaras, overlay
Numerica	NumPy	Operacoes sobre heatmaps e mascaras binarias
Hospedagem	Railway (Hobby plan)	Container Linux com Python 3.12, deploy via Nixpacks
Identificacao	Roboflow Serverless API	Modelo principal de classificacao

4.3. Treinamento e dados

Recurso	Detalhe
Ambiente	Google Colab (T4 GPU gratuita)
Framework	TensorFlow/Keras 3
Dataset	Corn-Maize-Leaf-Disease (PRMSU/PlantVillage) via Roboflow Universe
Volume	~4.180 imagens originais, 7.200 apos augmentation
Classes	4 (Blight, Common_Rust, Gray_Leaf_Spot, Healthy)
Acuracia (Roboflow)	95,8% no conjunto de teste

Acuracia (Keras local)	Aproximadamente 93-96% (varia por execucao)
------------------------	---

5. Arquitetura do Sistema

O sistema e dividido em tres camadas independentes que se comunicam por HTTPS/WSS. Essa separacao foi escolhida para permitir que cada camada evolua sem afetar as demais e para que o backend de IA pudesse ser servido em uma infraestrutura adequada (GPU/CPU dedicada), enquanto o app movel mantem-se leve.

5.1. Diagrama de fluxo logico

```
[Camera no celular] --(captura JPEG)--> [App React Native] --(base64 via
WebSocket)--> [Backend FastAPI no Railway] --(resize + base64)--> [Roboflow
Serverless]
```

Retorno: classificacao -> usuario.

Quando o usuario clica em **Auditar Modelo**:

```
[App] --(imagem base64)--> [Backend] --(forward pass + GradientTape)--> [Keras local]
--(heatmap + AL/AFS)--> [App] --(render)--> usuario.
```

5.2. Por que dois modelos separados?

A API serverless do Roboflow expoe apenas o forward pass (predicao final), o que torna impossivel calcular gradientes em camadas intermediarias. Como o Grad-CAM exige acesso ao **tape de gradientes**, mantemos um segundo modelo Keras local (MobileNetV2 treinado no mesmo dataset) cuja unica funcao e gerar o heatmap. Os dois modelos sao da mesma familia, treinados no mesmo dataset, o que garante coerencia entre identificacao e auditoria.

6. Fluxo do Usuario no App

6.1. Passo a passo

- **1. Home:** usuario abre o app e ve o cartao principal "Identificar Especie".
- **2. Camera:** ele captura uma foto da folha (preferencialmente em luz natural, sem sombras fortes).
- **3. Identificacao automatica:** o app envia a foto via WebSocket para o backend, que repassa ao Roboflow.
- **4. Resultado:** a tela ResultScreen exibe nome da doenca em PT-BR, confianca, descricao patologica e recomendacoes de manejo.
- **5. Auditoria (opcional):** usuario clica em "Auditar Modelo (Grad-CAM)" para validar cientificamente o resultado.
- **6. Visualizacao:** a tela AuditarModelo mostra heatmap sobreposto a imagem, AFS, AL, tempo de processamento e camada usada.
- **7. Salvar/Exportar:** usuario pode rotular o enquadramento, salvar no historico cientifico e exportar o overlay como PNG.

6.2. Telas principais

Tela	Funcao
HomeScreen	Ponto de entrada com CTA principal
CameraScreen	Wrapper sobre expo-camera, captura JPEG
ResultScreen	Identificacao + recomendacoes (Roboflow)
AuditarModeloScreen	Grad-CAM + metricas (Keras local)
InstitucionalScreen	Texto sobre o projeto / contato

7. Modelos de IA

7.1. Modelo de identificacao (Roboflow)

O modelo principal foi treinado na plataforma Roboflow utilizando ResNet50 com transfer learning a partir de pesos pre-treinados no ImageNet. A escolha por ResNet (He et al., 2016) se justifica pelo bom desempenho em classificacao de imagens com datasets de tamanho moderado (4-10 mil imagens) e pela capacidade dos blocos residuais de evitar o problema do *vanishing gradient* em redes profundas. Acuracia obtida no conjunto de teste: 95,8%.

7.2. Modelo de auditoria (Keras local)

Para o Grad-CAM utilizamos MobileNetV2 (Sandler et al., 2018) com transfer learning. A arquitetura foi escolhida por tres razoes: **(i)** tamanho reduzido (~14 MB) compativel com o limite de memoria do plano Hobby do Railway; **(ii)** presenca explicita de blocos convolucionais nomeados (Conv_1, block_16_*) que facilitam a selecao da camada-alvo do Grad-CAM; **(iii)** performance comparavel a redes maiores em datasets de folhas (varios papers de plant disease usam MobileNet).

Arquitetura final usada:

```
Input (224x224x3)
-> mobilenet_v2.preprocess_input (normalizacao [-1, 1])
-> MobileNetV2(include_top=False, weights='imagenet')
-> GlobalAveragePooling2D()
-> Dropout(0.5)
-> Dense(4, activation='softmax')
```

Treinamento em duas fases: (1) 15 epocas com base congelada, lr=1e-3; (2) 10 epocas de fine-tune das ultimas 50 camadas, lr=1e-4.

7.3. Dataset

Utilizamos o dataset **Corn-Maize-Leaf-Disease** publicado pela Pampanga State Agricultural University (PRMSU) e disponivel no Roboflow Universe. As imagens originais sao do projeto *PlantVillage* (Hughes & Salathe, 2015), referencia em pesquisas de plant disease detection. O dataset tem 4.186 imagens distribuidas entre as quatro classes, com particao padrao em train/valid/test.

8. Explainable AI (XAI) - Conceitos Essenciais

Explainable AI é um campo da IA dedicado a tornar as decisões de modelos de aprendizado de máquina compreensíveis para seres humanos. Em redes neurais profundas (especialmente CNNs), as decisões são tomadas a partir de milhões de parâmetros que interagem de forma não-linear, o que torna impossível entender o porquê da resposta apenas inspecionando os pesos.

O termo "explainable" se opõe a "black-box". Segundo Adadi & Berrada (2018), as técnicas de XAI buscam responder pelo menos uma das três perguntas: (1) *Por que o modelo tomou essa decisão?* (2) *Como ele tomaria uma decisão diferente?* (3) *Quando o modelo erra sistematicamente?* O AgroVision se concentra na primeira pergunta, usando uma técnica **post-hoc** e **baseada em gradientes**: o Grad-CAM.

8.1. Taxonomia rápida

- **Intrínseca vs post-hoc**: modelos intrinsecamente explicáveis (árvores, regressão linear) vs métodos aplicados depois (LIME, SHAP, Grad-CAM).
- **Global vs local**: explicar o modelo como um todo vs explicar uma predição individual. Grad-CAM é **local**.
- **Model-specific vs model-agnostic**: Grad-CAM é específico para CNNs; LIME e SHAP são agnósticos.

9. Grad-CAM: Mapeamento de Atenção Visual

Gradient-weighted Class Activation Mapping (Grad-CAM) foi proposto por Selvaraju et al. em 2017 (ICCV). É uma generalização do CAM original (Zhou et al., 2016) que removeu a limitação de exigir uma arquitetura específica com GAP no final.

9.1. Intuição

A última camada convolucional de uma CNN contém mapas de características (feature maps) que preservam informação espacial. Cada canal desses mapas é sensível a um padrão visual diferente (textura, borda, formato). O Grad-CAM pergunta: *"para a classe predita, quanto cada canal contribuiu? E onde esses canais estão mais ativos?"* A resposta combinada produz um mapa de calor que indica quais regiões da imagem foram decisivas.

9.2. Formulação matemática

Seja:

```
Ak(i,j) = ativação do canal k na posição (i,j) da última camada convolucional  
yc = score (logit ou probabilidade) da classe c  
Z = número de pixels da feature map (largura x altura)
```

Passo 1 - calcular o peso de importância de cada canal:

$$\alpha_{k^c} = (1/Z) * \sum_i \sum_j (\partial y^c / \partial A^k(i,j))$$

Passo 2 - somar os mapas ponderados e aplicar ReLU:

$$L_GradCAM^c(i,j) = \text{ReLU}(\sum_k \alpha_k^c \cdot A^k(i,j))$$

O ReLU final descarta contribuicoes negativas (pixels que *diminuiriam* a confianca da classe). O mapa resultante e normalizado para o intervalo [0, 1] e aumentado (upsample) para o tamanho da imagem original.

9.3. Implementacao no projeto

No arquivo **api/xai/gradcam.py** a funcao **generate_gradcam** usa **tf.GradientTape()** para registrar o forward pass, obtem o gradiente do score da classe em relacao aos feature maps da camada-alvo (por padrao, **Conv_1** do MobileNetV2), faz o pooling global dos gradientes para obter os pesos α_k , multiplica pelos feature maps e aplica ReLU + normalizacao.

10. Metricas Quantitativas: AL e AFS

O Grad-CAM por si so e **qualitativo**: produz uma imagem que o ser humano olha e interpreta. Para criar uma **auditoria quantitativa**, o AgroVision introduz duas metricas calculadas sobre o heatmap em conjunto com uma **regiao de interesse (ROI)** definida por uma bounding box.

A ideia central: dado que o heatmap mostra *onde o modelo olhou*, e que a bbox mostra *onde o modelo deveria olhar* (a regiao da folha que importa), podemos medir o quanto essas duas regioes coincidem. Se o modelo olha para fora da folha (fundo, sombra, mao do usuario), isso e um sinal de aprendizado espurio.

10.1. Attention Leakage (AL)

Attention Leakage (literalmente, "vazamento de atencao") mede a fracao da atencao do modelo que esta **fora** da regiao de interesse. Quanto maior o AL, mais o modelo se distraiu com elementos irrelevantes.

$$AL = \Sigma(H \times (1 - M)) / \Sigma(H)$$

onde:

H = heatmap binarizado pelo threshold tau (apenas pixels com atencao $\geq$ tau contam)

M = mascara binaria da regiao de interesse (1 dentro da bbox, 0 fora)

10.2. Attention Focus Score (AFS)

Attention Focus Score e o complemento do AL: mede a fracao da atencao que esta **dentro** da regiao de interesse. E uma metrica de qualidade: AFS alto = modelo focado.

$$AFS = 1 - AL = \Sigma(H \times M) / \Sigma(H)$$

10.3. Threshold e por que ele importa

Antes de calcular AL e AFS, o heatmap passa por um **threshold** (limite minimo de intensidade) configuravel - por padrao tau = 0.5. Pixels com ativacao abaixo desse valor sao zerados. Isso evita que ruido de fundo (pixels com ativacao 0.05 espalhados pela imagem) contamine o calculo, garantindo que so a atencao **relevante** (acima do threshold) seja contabilizada.

10.4. Implementacao real

```
# api/xai/metrics.py
active = np.where(heatmap >= threshold, heatmap, 0.0)
inside = float((active * mask).sum())
outside = float((active * (1 - mask)).sum())
total = inside + outside

AL = outside / total
AFS = 1 - AL
```

10.5. Como a bbox e definida

- **Fixed (padrao):** bbox centralizada cobrindo 80% da imagem. Assume que o usuario enquadrou a folha minimamente bem.
- **Green segmentation:** segmenta tons de verde no espaco HSV e usa a bounding box do maior blob como ROI. Util quando o enquadramento e ruim.
- **Manual:** usuario informa coordenadas (modo cientifico).

10.6. Casos praticos (interpretacao)

Caso 1 - Modelo focado e correto

AFS = 92%, AL = 8%, predicacao = Common_Rust (confianca 99%). Heatmap concentra-se exatamente sobre as pustulas marrom-alaranjadas tipicas da ferrugem, dentro da folha. **Interpretacao:** o modelo aprendeu o feature visual correto e nao se distraiu com bordas/fundo. Confianga alta na resposta.

Caso 2 - Modelo correto mas com atencao dispersa

AFS = 51%, AL = 49%, predicacao = Ferrugem-Comum (confianca 99%). Heatmap se distribui pela folha mas tambem ativa bordas e parte do fundo. **Interpretacao:** a classificacao esta correta, mas o modelo usou tambem informacao contextual (cor de fundo, textura ao redor). Para o agronegocio, ainda e aceitavel - mas indica que o modelo se beneficiaria de mais amostras com fundos variados.

Caso 3 - Modelo "acertou por sorte"

AFS = 25%, AL = 75%, predicacao = Healthy (confianca 87%). Heatmap concentra-se quase todo fora da folha (canto da imagem). **Interpretacao:** mesmo que a predicacao final possa ser correta, o modelo nao se baseou em caracteristicas da folha. Esse e um caso de **shortcut learning** (Geirhos et al., 2020). O sistema deve sinalizar a resposta como nao confiavel, apesar da confianca alta.

Caso 4 - Modelo errado e com atencao errada

AFS = 18%, AL = 82%, predicacao = Blight (confianca 65%) mas a folha era saudavel. Heatmap ativa principalmente uma sombra escura no canto inferior. **Interpretacao:** o modelo confundiu a sombra com lesao. AL alto + confianca media = forte indicio de que a resposta deve ser descartada. Idealmente, o app sugere ao usuario recapturar a foto.

10.7. Por que essas duas metricas e nao outras?

Existem metricas tradicionais de qualidade de explicacao na literatura (Pointing Game, Deletion/Insertion - Petsiuk et al., 2018; Sanity Checks - Adebayo et al., 2018). Algumas envolvem perturbacoes iterativas custosas, outras exigem ground-truth de saliencia (que exigiria anotacao manual pixel-a-pixel das lesoes - inviavel no nosso escopo).

AL e AFS sao deliberadamente **simples**: requerem apenas uma bbox por imagem (que pode ser inferida automaticamente ou fixada) e uma comparacao set-theoretic. Sao computaveis em milisegundos, interpretaveis e ja capturam o sinal mais importante: *onde a atencao do modelo esta concentrada*. A simplicidade tambem favorece a comunicacao com o agronomo leigo em IA, que entende facilmente "o modelo olhou 80% da hora pra dentro da folha".

11. Funcionalidades do Aplicativo

11.1. Identificacao

Fluxo principal. Captura ou seleciona uma foto, envia ao backend, exibe doenca identificada, confianca, descricao da patologia e recomendacoes de manejo. Suporte a quatro classes: Helmintosporiose, Ferrugem-Comum, Mancha-Cinzenta, Milho Saudavel.

11.2. Auditoria (Grad-CAM + AL/AFS)

Tela acessada pelo botao "Auditar Modelo" na ResultScreen. Executa Grad-CAM no backend, renderiza heatmap sobre a imagem, exibe os valores numericos de AL e AFS com barra de progresso colorida (verde $\geq 70\%$, amarelo entre 40-70%, vermelho $< 40\%$ para AFS).

11.3. Rotulagem de enquadramento

Dentro da auditoria, o usuario pode classificar a imagem como bem enquadrada / mal enquadrada / nao classificada. Esse rotulo e salvo junto com a analise, criando um conjunto de dados rotulados para estudos futuros de robustez do modelo.

11.4. Historico cientifico

Todas as analises auditadas e salvas vao para um banco SQLite local. Estrutura permite consulta posterior, geracao de estatisticas e exportacao em batch.

11.5. Exportar resultado

Apos salvar, o usuario pode exportar o heatmap como PNG via expo-sharing (compartilhamento nativo do iOS/Android). Util para documentar diagnosticos ou compartilhar com um agronomo consultor.

11.6. Modulo institucional

Tela com informacoes sobre o projeto, autores, contato e referencias academicas. Funciona tambem como vitrine cientifica caso o app seja apresentado publicamente.

12. Limitacoes e Trabalhos Futuros

- **Dois modelos paralelos:** identificacao usa Roboflow, auditoria usa Keras local. Sao da mesma familia (treinados no mesmo dataset), mas tem pesos diferentes - a auditoria explica o que o modelo Keras olharia, nao necessariamente o que o Roboflow olhou.
- **Bbox padrao e generica:** a estrategia "fixed" cobre 80% central da imagem. Se a folha estiver descentralizada ou a foto incluir muito fundo, o AL/AFS pode ser pessimista. Trabalho futuro: segmentacao automatica via U-Net ou SAM.

- **Apenas quatro classes:** o sistema não detecta deficiências nutricionais, pragas (insetos) ou outras doenças além das três modeladas. Expansão requer novo dataset.
- **Sem fallback offline:** a versão atual depende de conexão para classificar. TFLite local foi removido durante o pivot para milho - pode ser re-introduzido em uma próxima versão.
- **AL/AFS não consideram a forma do heatmap:** dois heatmaps com mesma quantidade de atenção mas distribuições diferentes (concentrada vs dispersa) tem o mesmo AFS. Trabalho futuro: incorporar métricas de entropia espacial.

13. Referencias Bibliograficas

- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D.** Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV 2017.
- Zhou, B., Khosla, A., Lapedriza, A., Oliva, A., & Torralba, A.** Learning Deep Features for Discriminative Localization (CAM). CVPR 2016.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.** MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR 2018.
- He, K., Zhang, X., Ren, S., & Sun, J.** Deep Residual Learning for Image Recognition (ResNet). CVPR 2016.
- Adadi, A., & Berrada, M.** Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence (XAI). IEEE Access, 2018.
- Hughes, D. P., & Salathe, M.** An open access repository of images on plant health to enable the development of mobile disease diagnostics (PlantVillage). arXiv, 2015.
- Geirhos, R., et al.** Shortcut Learning in Deep Neural Networks. Nature Machine Intelligence, 2020.
- Adebayo, J., et al.** Sanity Checks for Saliency Maps. NeurIPS 2018.
- Petsiuk, V., Das, A., & Saenko, K.** RISE: Randomized Input Sampling for Explanation of Black-box Models. BMVC 2018.
- Pan, S. J., & Yang, Q.** A Survey on Transfer Learning. IEEE Transactions on Knowledge and Data Engineering, 2009.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P.** Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE, 1998.
- Embrapa Milho e Sorgo.** Doencas da cultura do milho. Circular Tecnica, varios anos.
- Pampanga State Agricultural University (PRMSU).** Corn-Maize Leaf Disease Dataset. Roboflow Universe, 2023.

Documento gerado automaticamente para fins de estudo e referencia interna do projeto AgroVision. Para citacao academica formal, consulte o repositorio oficial e o trabalho completo de conclusao de curso.